STAGE 2 – PRACTICAL ENGINEERING CHALLENGE

TEAM ID: LRC-26-0675	TEAM NAME: GATE KEEPERS

Problem Statement: Industrial Manufacturing Process Automations

Industrial manufacturing depends on highly controlled liquid-handling systems to deliver process fluids reliably from a source or bulk supply to downstream process equipment. Depending on the application, these fluids may include **ultrapure fluids, process chemicals, solvents, developers, or CMP-related fluids**. Industry standards explicitly address liquid-chemical distribution, high-purity piping, bulk chemical distribution, system qualification, and UPW system operation and monitoring.

In such systems, **flow is a fundamental process variable**. An incorrect flow rate can indicate or cause a restriction, incorrect valve state, inadequate pumping performance, abnormal process demand, or another hydraulic deviation. The consequence may be unstable process conditions, incorrect chemical delivery, equipment malfunction, or loss of process performance.

This challenge therefore focuses on a fundamental industrial facilities and process-engineering task: **To establish a defined process-fluid flow, measure it reliably, detect a deviation, diagnose its cause, and restore the process to the required operating condition.**

Engineering Challenge

As you are part of a multidisciplinary engineering team responsible for designing, building and validating a small-scale automated liquid-flow system represented of industrial process handling system.

CASE 1 – CONTROL + MEASUREMNT

Problem: Develop a system that manages the changing level of fluid in the sump automatically (when there is a sudden external fluid charging to the sump) and while accurately measuring and totalizing the fluid discharged from the sump using the flow transmitter.

CASE 2 – FLUSHING +SHUTDOWN

Problem: Develop an alternative operating configuration that allows flush water to pass through the system while all three pumps operate simultaneously, and then safely transition the system to an OFF condition when the flush-water supply has been exhausted.

Given Process Parameters

Pipe diameter =	**8mm**
Fluid density (ρ) =	**1000kg/m^3**
Dynamic Viscosity (μ) =	**1x10^{-3}Pa.s**
Required Flow range =	**1-6 L/min**
Nominal target flow for the live challenge =	**3L/min**

Available Hardware

Mechanical Hardware	Electronic Hardware
1. Water Tank – 3 Quantity	1. Custom PCB
2. Pumps – 3 Quantity	2. ESP32 NodeMCU module
3. FT-01 Flow Transmitter – 01 Quantity	3. Pump Driver-1
4. Ball valve – 3 Quantity	4. Pump Driver-2
5. ON/OFF/XV Valve – 1 Quantity	5. Pump Driver-3
6. 10-mm OD tubing/blue – 9 meters/1 meter	6. 12 V DC power input (**SMPS**)
7. Microcontroller/PCB/ data-acquisition software	7. 12 VDC Solenoid-valve interface
8. Required electrical/ electronic accessories	8. Flow-sensor interface
	9. Float-switch interface
	10. ON/OFF-switch interface
	11. Pump-driver PWM/enable interfaces
	12. On/Off Switch

The team must determine the appropriate arrangement, connections and measurement method.

- Assemble and commission the liquid-flow system using the components provided.
- Measure and characterize the flow using FT-01
- Interface FT-01 with the provided **data-acquisition system**
- Experimentally determine the relationship between sensor output and actual flow.
- Characterize the effect of valve position on flow rate. Record and analyse flow vs time data
- Maintain target flow of 3L/min
- Use graphs and measured data to justify engineering decisions
- Design an appropriate hydraulic arrangement and P&ID

TEAM INSTRUCTIONS

- As a team go through the list provided to read about each component that has been Listed.
- The PCB is a 12 V DC and the SMPS connections should be made carefully.

TASK 1: HYDRAULICS SYSTEM DESIGN

Q1.1 Design and build the hydraulic path with the flow transmitter on the recirculation line. Submit/sketch a labelled P&ID below by showing the pumps, Needle valves, flow transmitter, flow direction, Tank, sump, tube diameter and measurement points.

CASE 1 – Process Fluid Transfer & Automatic Level Protection

Objective: Set-up and operate a closed-loop process- fluid handling system using the available tank, pump, valves flow transmitter, sump, float sensor and control PCB. The System must continuously transfer process fluid recirculation system by connecting the 10L fluid tank with a On/Off valve to a pump-01 and the pump-01 goes to the inlet of sump 4.2L tank. The Sump 4.2L outlet line is connected to another pump-02 followed by a flow transmitter that measures the total flow back to the 10L fluid tank. The Pump-01 runs on 50% duty cycle and Pump-02 runs on 100% duty cycle. **Calibrate the flow transmitter** and demonstrate correct fluid circulation, if there is sudden introduction of external fluid into the Sump tank then Pump-01 should stop while pump – 02 should still runs till fluid level reduces in the sump. After the fluid reduction in the SUMP 4.2L tank the Pump-01 should turn on and the process must continue. **Also, demonstrate Flow vs Time trends in mL/s, total transferred Liters back to the 10L tank** (share Screenshots flow in L/min). **Share the code**. The case-1 can be turned on using the available on/off Switch, to start the system.

CASE 2 – Flush/Line Conditioning Sequence

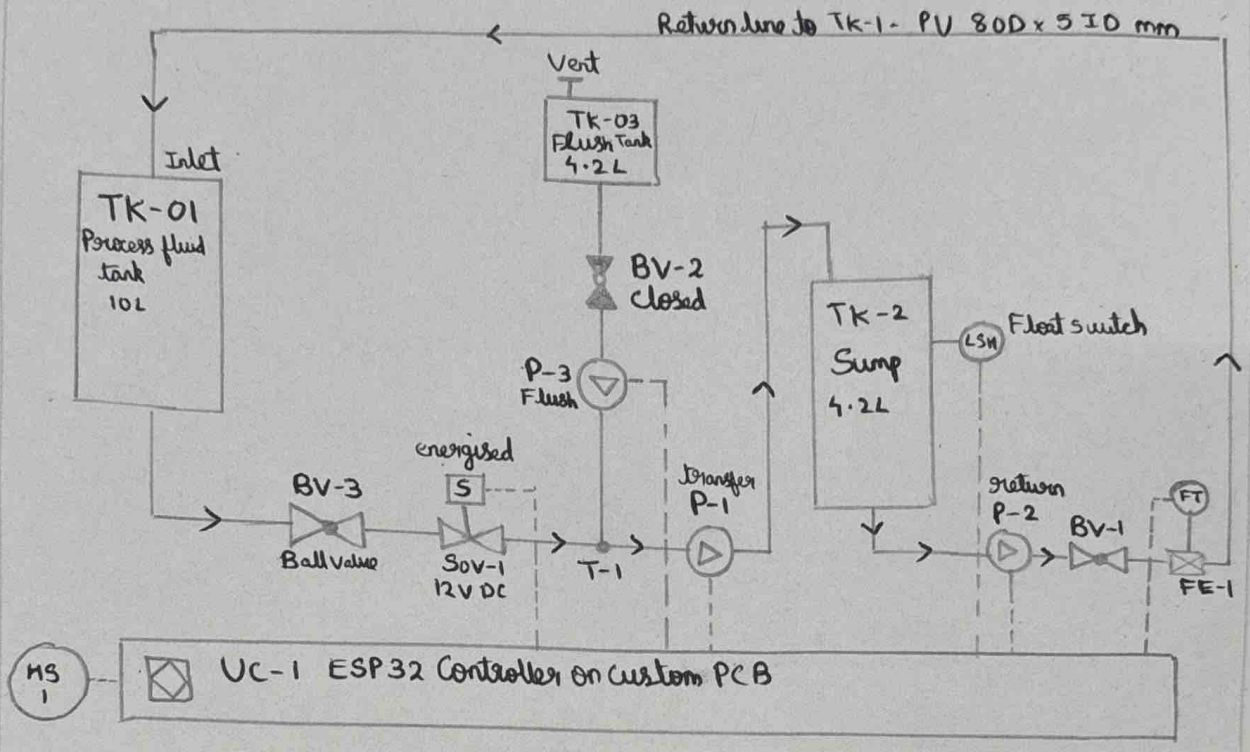
Objective: After successfully completing case 1, initiate the flush water in the sequence using the **equal Tee.** The On/Off valve must be switched off in order to isolate the 10L process fluid tank. The flush Fluid tank (4.2L) is used as a temporary fluid source to flush out the process fluid or any debris from the system. Before the start of this process make sure you open the manual ball valve on the fluid flush line before Pump-03 starts. The entire process of case -1 and case - 2 should run in a single go without manual intervention except opening of ball valve before the Pump-03 starts. **The System must stop as soon as the tank flush fluid 4.2L tank finishes.** The system must be automated without human interventions (only ball valve can be opened manually before the start of flush line sequence)

SUBMIT:

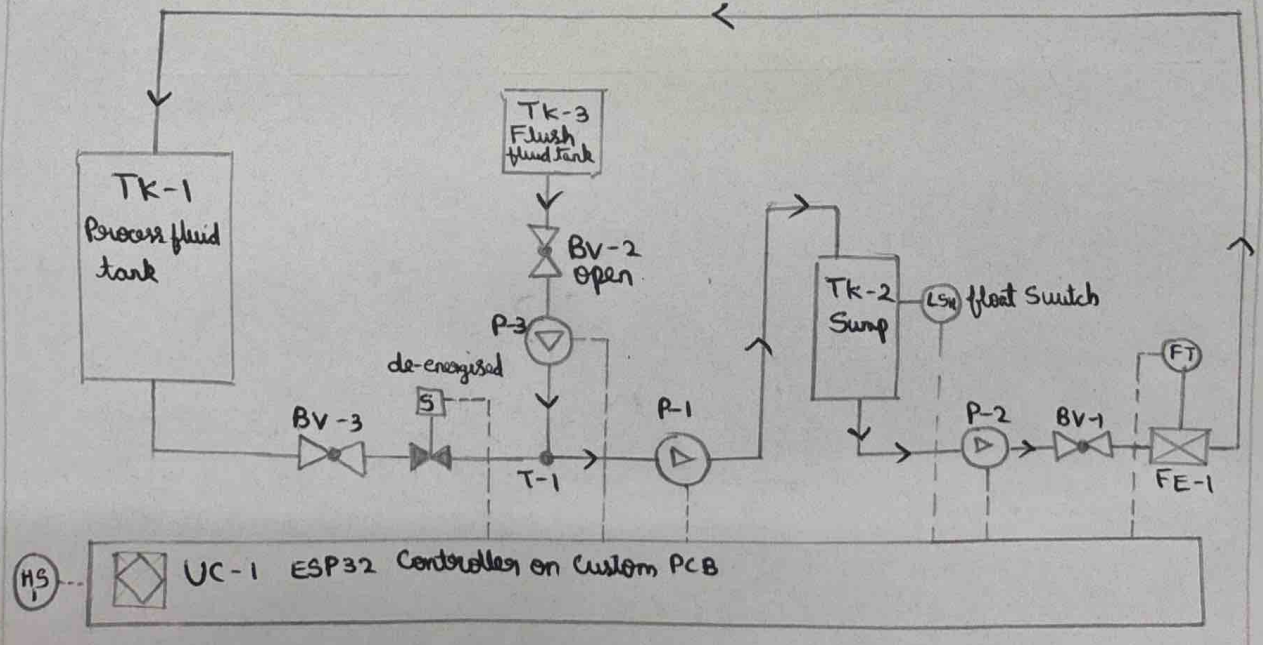
A. Submit: 10-15 min Video and Photograph of the completed setup, with identification of each component and explain the direction of flow in the video verbally. Demonstrate both Case - 1 and Case – 2.

B. Final P&ID Block diagram

Case 1 : Process fluid transfer and automatic level protection



Case 2 : Flush and line conditioning, then automatic shutdown



Q1.2 After setting-up the system, now operate it at approximately at 1L/min, 3L/min & 6L/min. Compare the theoretical velocity with the actual operating behaviour.

WORKING / COMPARISON

Table-1: Theoretical Vs actual velocity

P-2 duty	Theoretical Q (L/min)	Theoretical V (m/s)	Measured Q (L/min)	Measured Q (mL/s)	Actual V (m/s)	Actual Vs Theory
25%	0.5	0.42	0.36	6	0.31	-28%
50%	1	0.85	1.02	17	0.87	+2%
100%	2	1.7	1.95	32.5	1.66	-2.5%

Table-2. Flow regime and friction loss at measured flows.

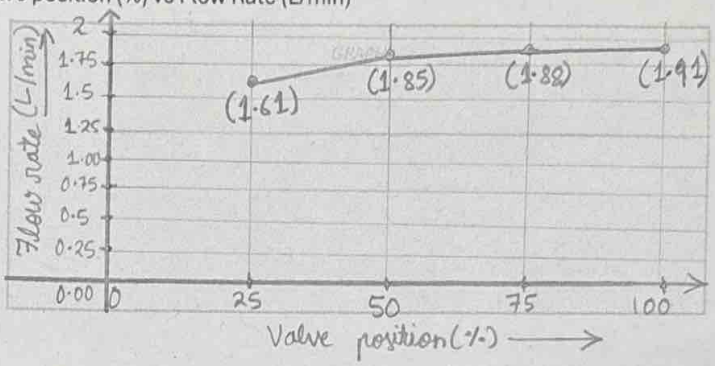
P-2 duty	Re, 5mm	Regime, 5mm	ΔP, 5mm (Pa/m)	Re, 8mm	Regime, 8mm	ΔP, 8mm (Pa/m
25%	1530	Laminar	about 390	955	Laminar	about 60
50%	4330	Turbulent	about 2920	2710	Transitional	about 310
100%	8280	Turbulent	about 9080	5170	Turbulent	about 970

NOTE: The maximum operable flow rates for the given pumps is around 2L/min, as opposed to the claimed 6L/min on the question. The parts are answered in accordance to the derived duty cycles rather than the mentioned flow rates

TASK 2: FLOW RESPONSE TO VALVE POSITION

Q 2.1 Set the ball valve to approximately 25%,50%,75% & 100%. At each position, allow the flow to stabilize and record the flow rate. Create the following graph of valve position (%) vs Flow Rate (L/min)

VALVE POSITION	FLOW RATE
25%	1.61 L/min
50%	1.85 L/min
70%	1.88 L/min
100%	1.91 L/min



Q2.2 Determine the experimental calibration equation: $Q = af + b$, where Q = flow rate in L/min, f = pulse frequency in Hz, a = experimental calibration coefficient and b = offset.

Report:

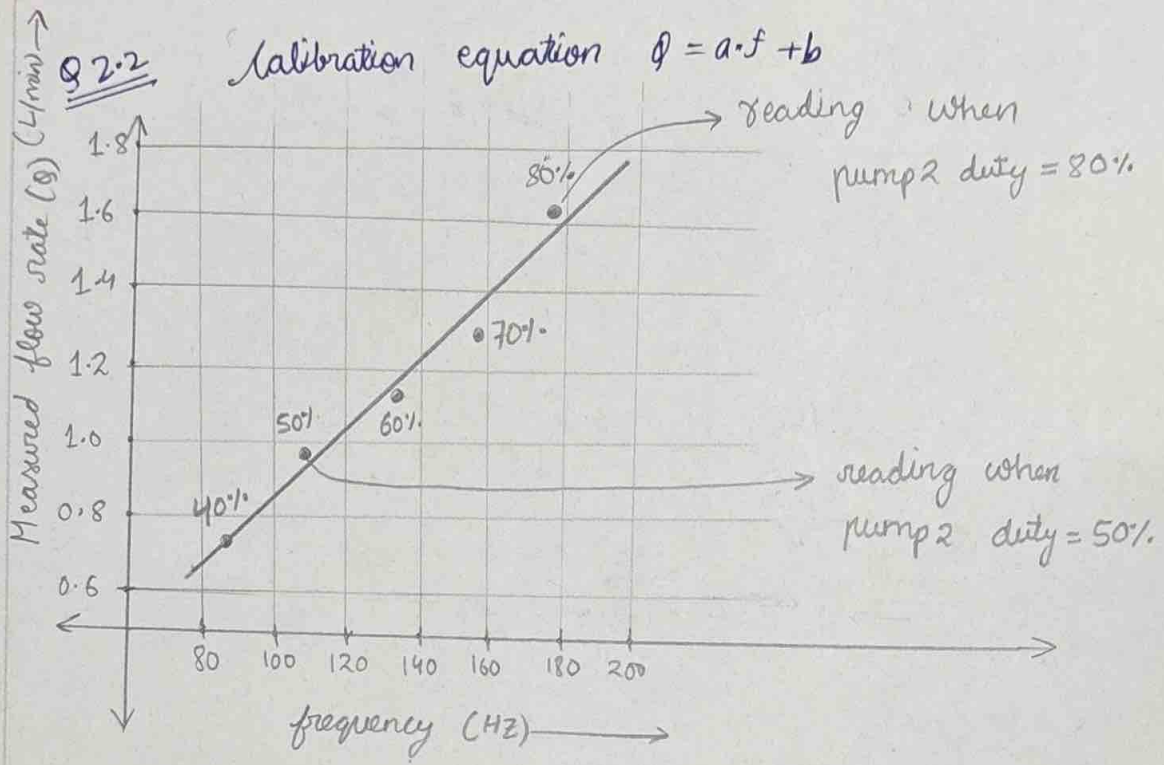
R^2 =	0.991
a =	0.009213 L/min per Hz
b =	−0.0672 L/min
Maximum calibration error =	3.6%
Average calibration error =	1.8%

Q2.3 FT-01 produces pluses corresponding to water flow. Run the system at least for 5 different flow conditions within the available operating range (Hint: use formulas f=N/t and Q=V/t). For each condition, record the following:

Trial	Pulses	Time(s)	Frequency (Hz)	Collected Volume (L)	Actual Flow (L/min)
1	5658 ~~5640~~	65.19	86.79	0.800	0.736
2	5489	49.67	110.51	0.800	0.966
3	5939	44.99	132.00	0.855	1.140
4	5305	33.64	157.69	0.750	1.337
5	5307	29.66	178.91	0.800	1.618

CALCULATIONS / OBSERVATIONS

Q 2.2 Calibration equation $Q = a \cdot f + b$



The above plot is drawn from the readings taken in Q2.3

We use this ~~pt~~ readings for calibration purpose.

→ For calibration I am using "least Square Means" for picking the straight line.

$$a = \frac{n \sum f_i Q_i - \sum f_i \sum Q_i}{n \sum f_i^2 - (\sum f_i)^2}, \quad b = \frac{\sum Q_i - a \sum f_i}{n}$$

$$R^2 = 1 - \frac{\sum (Q_i - \hat{Q}_i)^2}{\sum (Q_i - \bar{Q})^2}$$

$$\text{calibration error } (e_i) = \frac{|\hat{Q}_i - Q_i|}{Q_i} \times 100\%$$

Maximum calibration error = $\max(e_i)$

$$\text{mean calibration error} = \frac{\sum e_i}{n}$$

TASK 3: HYDRAULIC SYSTEM THEORY AND FAULT ANALYSIS

Q3.1 The effective liquid volume inside a section of tubing is 0.5L, while the operating flow rate is 2.5L/min. The hydraulic residence time is given by $T_r = Q/V$. Calculate the residence time in seconds, also answer the following questions:

1. What does residence time physically represent in this system?
2. What happens to residence time if the tubing volume is doubled while the flow rate remains constant?
3. What happens to residence time if the flow rate is doubled while the tubing volume remains constant?
4. Why is residence time important when interpreting a sudden change in measured flow?

ANSWER

Residence time is volume divided by flow $\left(T_r = \frac{V}{Q}\right)$. So according to the corrected formula

$$\rightarrow T_r = \frac{V}{Q} = \frac{0.5L}{2.5L/min} = 0.2min = 12s$$

① It represents the average time a fluid element spends inside that tube section, from entering to leaving.

② Since $T_r = \frac{V}{Q}$ and V doubles and Q stays the same, residence time becomes twice the initial value ($T_r = 24s$)

③ Here Q doubles & V stays same → $T_r = 6s$

④ A change in the flow reaches FT-01 almost at once, but anything carried by water (like air bubbles or debris) arrives only about T_r later. So a sudden step in flow points to the pump, a valve, a blockage, on the sensor, while a disturbance that shows up about T_r after an upstream event was carried by the fluid. T_r also sets how long a flush must run to clear a section.

Q3.2 The following observation are obtained from the same system:

CONDITION	PUMP	VALVE	FT-01 PULSE SIGNAL	MEASURED FLOW
A	ON	Open	Normal	Normal
B	ON	Partially closed	Reduced	Reduced
C	ON	Open	Zero	Zero
D	OFF	Open	Zero	Zero
E	ON	Open	Intermittent	Fluctuating

For each of the condition, identify the most likely physical or instrumentation condition. Then answer: Which condition is most difficult to distinguish using FT-01 alone, and what additional information would you require to distinguish the fault? (Hint: Teams must consider possibilities such as: Valve restriction, pump failure, hydraulic blockage, sensor/signal failure, unstable flow etc)

ANSWER

A) This condition represents normal operation

B) This condition is caused due to restricted valve. The added valve loss lowers the flow. It is an expected response, not a fault.

C) Here, there is no flow even when pump is on. It is caused by either: hydraulic blockage (eg: due to any closed valve), pump failure (issue with motor, or driver) or sensor/signal failure (eg: jammed rotor or broken wire)

D) This is a normal case, since the pump is off and sensor gives no false pulses

E). Unstable flow: due to either air entering the suction, cavitation, or a loose/noisy signal wire

- Hardest to distinguish with FT-01 alone would be C as zero pulses look the same whether pump failed, line is blocked, or the sensor failed
- Extra information needed: (i) pump motor current: high = blockage, near zero = pump failure, normal = sensor fault
  (ii) pressure after the pump
  (iii) whether sump level is changing
  (iv) a check of sensor's 5V supply & signal.

HARDWARE LIST

Mechanical Hardware

Component	Description	Main Use
Ball Valve (BV-01)	A manually operated valve with a spherical shut-off mechanism.	Used to **manually isolate or permit fluid flow** in the pipeline. It is useful for maintenance, testing, and flow isolation.
10-Litre Fluid Tank (Fluid Tank-01)	Transparent storage tank with approximately 10 L capacity.	Used as the **main fluid reservoir** for storing process fluid. The transparent body also allows the fluid level and condition to be visually monitored.
5-Litre Sump	Small transparent reservoir labelled **SUMP**, approximately 4.5 L.	Used as a **collection/storage vessel** for fluid returning to or entering the sump section of the system.
4.2-Litre Flush Fluid Tank / Flush Line	The transparent vessel labelled **FLUSH FLUID** is approximately 4.5 L.	Provides **flush fluid** for cleaning or flushing the main pipe circuit when required.
ON/OFF Valve (OV-01)	Electrically operated valve/solenoid-type valve shown in the setup.	Used for **automatic control of fluid flow**, allowing the controller to turn a particular flow path on or off.
Pumps -Pump-1, Pump-2, Pump-3	Motor-driven fluid pumps. Three pumps are shown in the setup.	Used to **circulate and transfer fluid** between different sections of the system. Each pump can be independently controlled.
8 mm Process Pipe	Flexible blue tubing with approximately 8 mm process-line size.	Used as the **main fluid-flow path** connecting tanks, valves, pumps, tees, and other components.
10 mm × 8 mm Reducers	Reducer fittings that transition between a 10 mm connection and an 8 mm pipe.	Used to **adapt the pump connection size to the 8 mm process pipe. Mainly used for Pumps**
8 mm Connectors	Push-fit/barbed-type fittings compatible with the 8 mm tubing.	Used to **join pipes to components** and to make detachable connections between sections of tubing.
8 mm Equal Tee	T-shaped connector having three equal 8 mm ports.	Used to **split one 8 mm fluid line into two branches or combine two branches into one line.**

Electronic Hardware

Electronic Component	Identification / Function	Use
PCB -Printed Circuit Board	Custom control board	Provides the electrical interconnection between the **12 V power supply, ESP32 controller, sensors, valves, indicators, and pump-driver interfaces.**
ESP32 / NodeMCU ESP32S module (U7)	Main microcontroller	Acts as the **central controller**. It provides GPIO/PWM signals for controlling outputs and receiving signals from sensors/switches.

Electronic Component	Identification / Function	Use
12 V DC Input Connector	Marked **12V INPUT**	Connection point for the external 12 V DC power supply.
12 V Output Connectors	P1, P2, P3 and additional 12 V output terminals	Provide 12 V power connections for external loads such as the pump-driver/motor circuits.
Motor Driver Interfaces -U2, U3, U4	Three multi-pin driver connectors shown on the PCB	Provide dedicated interfaces for **Pump-1, Pump-2 and Pump-3 driver/control connections**.
Solenoid Valve Connector	Labelled **SOLENOID_VALVE -S/V/G**	Interface for controlling/connecting the solenoid or ON/OFF valve.
Flow Sensor Connector	Labelled **FLOW_SENSOR - S/V/G**	Interface for the water-flow sensor; supplies power and receives the sensor signal. **THE FLOW SENSOR SHOULD BE CALIBRATED**
Float Switch Connector	Labelled **FLOAT_SWITCH -FSW/GND**	Interface for a float/level switch used to detect a water-level condition.
ON/OFF Switch Connector	Labelled **ON/OFF_SWITCH -SW/GND**	Interface for an external manual ON/OFF switch.
12V DC SMPS	**12V DC SMPS**	For Power Supply to the board

Flow Skid Code Listing

Lam Research Challenge 3.0, Stage 2 · Team GateKeepers

main.cpp

C++ · 804 lines

```cpp
/*
 * Lam Research Challenge 3.0, Stage 2 - flow skid controller
 * ESP32 (NodeMCU-32S) on the LAM Stage 2 PCB, PlatformIO + Arduino
 *
 * Case 1: the button opens SOV-01 and runs P-01 at 50 % and P-02 at 100 %.
 *   If the sump float goes high, P-01 stops until the level has been low for 3 s.
 * Case 2: SOV-01 closes and P-01 stops while BV-02 is opened by hand, then
 *   P-01, P-02 and P-03 all run.
 * The button steps IDLE -> Case 1 -> Case 2 -> DONE. Without a press, Case 1
 * ends after case1Time and Case 2 after case2Time (counted from P-03 start).
 * "stop" on serial stops everything.
 *
 * Wiring
 *   U2/U3/U4 go to the IBT_2 (BTS7960) boards. P-01 and P-03 get PWM on RPWM
 *   with LPWM low. P-02's PWM lines don't reach its driver (no R_PWM2 trace on
 *   the PCB, L_PWM2 open), so its RPWM pin is jumpered to the driver's 5 V,
 *   L_EN2 is held high and the speed PWM goes out on R_EN2 at 1 kHz.
 *   Pump red -> M+. Flow sensor: red V, black G, yellow S, mounted vertical
 *   with the flow going up.
 *
 * Serial 115200. Every logMs a CSV line:
 *   D,t_ms,state,pulses,f_Hz,Q_Lmin,Q_mLs,total_L,float_high,p1,p2,p3,sov
 * Lines starting with # are messages. tools/flow_logger.py saves and plots the
 * log. "help" lists the commands.
 *
 * Flow: Q [L/min] = a*f [Hz] + b. The defaults come from a least-squares fit of
 * five bucket trials (trial / fit commands); values saved with calsave or
 * fit save override them.
 */

#include <Arduino.h>
#include <Preferences.h>

// Pins
const uint8_t P1_RPWM = 33, P1_LPWM = 32, P1_REN = 26, P1_LEN = 13; // U2
const uint8_t P2_RPWM = 19, P2_LPWM = 18, P2_REN = 17, P2_LEN = 16; // U3
const uint8_t P3_RPWM = 25, P3_LPWM = 23, P3_REN = 22, P3_LEN = 21; // U4

const uint8_t SOLENOID_PIN = 27;
const uint8_t FLOW_PIN = 14;
const uint8_t BUTTON_PIN = 34; // 10k pull-up on the PCB
const uint8_t FLOAT_PIN = 35;  // 10k pull-up on the PCB
const uint8_t LED_PIN = 2;

const uint8_t CASE1_DUTY_P1 = 50;
const uint8_t CASE1_DUTY_P2 = 100;
const uint8_t FLUSH_DUTY_P1 = 100;
const uint8_t FLUSH_DUTY_P2 = 100;
const uint8_t FLUSH_DUTY_P3 = 100;

uint32_t case1Time = 100;
uint32_t bv02Window = 5;
uint32_t case2Time = 60;

const int FLOAT_HIGH = LOW;
const uint32_t FLOAT_DEBOUNCE_MS = 300;
const uint32_t FLOAT_CLEAR_MS = 3000;
const uint32_t BUTTON_DEBOUNCE_MS = 50;

const float DEFAULT_A = 0.009213;
const float DEFAULT_B = -0.0672;
const uint32_t GLITCH_US = 500;

```

```
const uint32_t PWM_FREQ = 20000;
const uint32_t P2_PWM_FREQ = 1000;
const uint8_t PWM_BITS = 10;
const uint32_t RAMP_MS = 600;

uint32_t logMs = 500;

const float KP = 8, KI = 6;
const float TARGET_FLOW = 3.0;

struct Pump
{
  uint8_t rpwm, lpwm, ren, len, channel;
  float target, duty;
};

Pump pumps[3] = {
    {P1_RPWM, P1_LPWM, P1_REN, P1_LEN, 0},
    {P2_RPWM, P2_LPWM, P2_REN, P2_LEN, 2},
    {P3_RPWM, P3_LPWM, P3_REN, P3_LEN, 4},
};
enum
{
  P1,
  P2,
  P3
};

uint8_t pwmPin(int i) { return i == P2 ? pumps[i].ren : pumps[i].rpwm; }

void writeDuty(int i, float pct)
{
  uint32_t duty = pct * ((1 << PWM_BITS) - 1) / 100 + 0.5f;
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcWrite(pwmPin(i), duty);
#else
  ledcWrite(pumps[i].channel, duty);
#endif
}

void setupPump(int i)
{
  Pump &p = pumps[i];
  uint8_t pins[] = {p.rpwm, p.lpwm, p.ren, p.len};
  for (uint8_t pin : pins)
  {
    pinMode(pin, OUTPUT);
    digitalWrite(pin, LOW);
  }
  uint32_t freq = i == P2 ? P2_PWM_FREQ : PWM_FREQ;
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(pwmPin(i), freq, PWM_BITS);
#else
  ledcSetup(p.channel, freq, PWM_BITS);
  ledcAttachPin(pwmPin(i), p.channel);
#endif
  writeDuty(i, 0);
}

void setPump(int i, float pct) { pumps[i].target = constrain(pct, 0.0f, 100.0f); }

void stopPumps()
{
  for (int i = 0; i < 3; i++)
  {
    pumps[i].target = pumps[i].duty = 0;
    writeDuty(i, 0);
    if (i != P2)
      digitalWrite(pumps[i].ren, LOW);
    digitalWrite(pumps[i].len, LOW);
  }
```

```
}

// soft start; stopping is immediate
void updatePumps()
{
  static uint32_t last = millis();
  uint32_t now = millis();
  float step = 100.0f * (now - last) / RAMP_MS;
  last = now;
  for (int i = 0; i < 3; i++)
  {
    Pump &p = pumps[i];
    p.duty = p.target > 0 ? min(p.target, p.duty + step) : 0;
    bool on = p.duty > 0;
    if (i != P2)
      digitalWrite(p.ren, on);
    digitalWrite(p.len, on);
    writeDuty(i, p.duty);
  }
}

bool sovOpen = false;

void setSolenoid(bool open)
{
  sovOpen = open;
  digitalWrite(SOLENOID_PIN, open);
}

volatile uint32_t pulseCount = 0, rawEdges = 0, lastPulseUs = 0;

void IRAM_ATTR onPulse()
{
  uint32_t now = micros();
  rawEdges++;
  if (now - lastPulseUs >= GLITCH_US)
  {
    pulseCount++;
    lastPulseUs = now;
  }
}

uint32_t readPulses()
{
  noInterrupts();
  uint32_t n = pulseCount;
  interrupts();
  return n;
}

float calA = DEFAULT_A, calB = DEFAULT_B;
float freqHz = 0, flowLpm = 0, totalL = 0;
uint32_t runPulses = 0, lastSampleMs = 0, lastPulses = 0;

bool fsRaw = false;
uint32_t fsHigh = 0, fsLow = 0, fsLastRaw = 0;

void sampleFlow()
{
  uint32_t now = millis();
  uint32_t n = readPulses();
  float dt = (now - lastSampleMs) / 1000.0f;
  freqHz = (n - lastPulses) / dt;
  flowLpm = freqHz > 0 ? max(0.0f, calA * freqHz + calB) : 0;
  totalL += flowLpm * dt / 60;
  runPulses += n - lastPulses;
  lastPulses = n;
  lastSampleMs = now;
}

bool buttonPressed()
```

```
{
  static int stable = HIGH, last = HIGH;
  static uint32_t changedAt = 0;
  int r = digitalRead(BUTTON_PIN);
  if (r != last)
  {
    last = r;
    changedAt = millis();
  }
  if (millis() - changedAt >= BUTTON_DEBOUNCE_MS && r != stable)
  {
    stable = r;
    return r == LOW;
  }
  return false;
}

bool floatHigh = false;

void readFloat()
{
  static int last = -1;
  static uint32_t changedAt = 0;
  int r = digitalRead(FLOAT_PIN);
  if (r != last)
  {
    last = r;
    changedAt = millis();
  }
  if (millis() - changedAt >= FLOAT_DEBOUNCE_MS)
    floatHigh = r == FLOAT_HIGH;
}

enum State
{
  IDLE,
  CASE1,
  FLUSH_PREP,
  FLUSH,
  DONE,
  TEST
};
const char *stateName[] = {"IDLE", "CASE1", "FLUSH_PREP", "FLUSH", "DONE", "TEST"};
State state = IDLE;
uint32_t stateStart = 0;

bool p1Held = false; // P-01 stopped by the float
uint32_t floatLowSince = 0;
bool holdFlow = false; // "hold 1": PI on P-02 during Case 1
bool piOn = false;
float setpoint = TARGET_FLOW, integral = 0;

uint32_t secondsInState() { return (millis() - stateStart) / 1000; }

void enterState(State s)
{
  state = s;
  stateStart = millis();
  Serial.printf("# STATE -> %s\n", stateName[s]);
  switch (s)
  {
  case IDLE:
  case DONE:
    stopPumps();
    setSolenoid(false);
    piOn = false;
    if (s == DONE)
      Serial.printf("# Sequence complete. Total transferred: %.3f L (%lu pulses). Press ON/OFF to run ag
ain.\n",
                    totalL, (unsigned long)runPulses);
    break;
```

```
  case CASE1:
    totalL = 0;
    runPulses = 0;
    setSolenoid(true);
    delay(300); // let SOV-01 open before the pumps pull on it
    p1Held = false;
    integral = 0;
    setpoint = TARGET_FLOW;
    setPump(P1, CASE1_DUTY_P1);
    setPump(P2, CASE1_DUTY_P2);
    setPump(P3, 0);
    Serial.printf("# CASE 1: SOV-01 open, P-01 %u%%, P-02 %u%%, runs %lus\n",
                  CASE1_DUTY_P1, CASE1_DUTY_P2, (unsigned long)case1Time);
    break;
  case FLUSH_PREP:
    setSolenoid(false);
    setPump(P1, 0);
    setPump(P3, 0);
    setPump(P2, 100); // empty the sump
    Serial.printf("# CASE 2: SOV-01 closed. OPEN BALL VALVE BV-02 NOW. P-03 starts in %lus (type 'next'
to start now)\n",
                  (unsigned long)bv02Window);
    break;
  case FLUSH:
    p1Held = false;
    setPump(P1, FLUSH_DUTY_P1);
    setPump(P2, FLUSH_DUTY_P2);
    setPump(P3, FLUSH_DUTY_P3);
    Serial.printf("# CASE 2: flushing, all three pumps ON for %lus. Press ON/OFF to stop early.\n",
                  (unsigned long)case2Time);
    break;
  case TEST:
    stopPumps();
    Serial.println("# TEST mode: use p1/p2/p3 <0-100>, sov 0|1, flow <L/min>|off. 'stop' to exit.");
    break;
  }
}

// P-03 only feeds P-01, so it stops with it
void handleFloat(uint8_t p1Duty, uint8_t p3Duty)
{
  if (floatHigh)
  {
    floatLowSince = 0;
    if (!p1Held)
    {
      p1Held = true;
      setPump(P1, 0);
      setPump(P3, 0);
      Serial.println("# SUMP HIGH: P-01 stopped, P-02 draining");
    }
  }
  else if (p1Held)
  {
    if (!floatLowSince)
      floatLowSince = millis();
    if (millis() - floatLowSince >= FLOAT_CLEAR_MS)
    {
      p1Held = false;
      setPump(P1, p1Duty);
      setPump(P3, p3Duty);
      Serial.println("# Sump level normal: P-01 restarted");
    }
  }
}

void runPI(float dt)
{
  float err = setpoint - flowLpm;
  float out = setpoint / 6 * 100 + KP * err + KI * (integral + err * dt); // 6 L/min ~ 100 %
  if (out > 0 && out < 100)
```

```
    integral += err * dt;
  setPump(P2, out);
}

void runSequence()
{
  if (state == CASE1)
  {
    handleFloat(CASE1_DUTY_P1, 0);
    if (secondsInState() >= case1Time)
      enterState(FLUSH_PREP);
  }
  else if (state == FLUSH_PREP)
  {
    static uint32_t shown = 0;
    uint32_t left = bv02Window > secondsInState() ? bv02Window - secondsInState() : 0;
    if (left != shown)
    {
      shown = left;
      Serial.printf("# P-03 starts in %lus - open BV-02\n", (unsigned long)left);
    }
    if (!left)
      enterState(FLUSH);
  }
  else if (state == FLUSH)
  {
    handleFloat(FLUSH_DUTY_P1, FLUSH_DUTY_P3);
    if (secondsInState() >= case2Time)
      enterState(DONE);
  }
}

void updateLed()
{
  uint32_t t = millis();
  bool on;
  switch (state)
  {
  case IDLE:
    on = t % 2000 < 100;
    break;
  case CASE1:
    on = true;
    break;
  case FLUSH_PREP:
    on = t % 200 < 100;
    break;
  case FLUSH:
    on = t % 1000 < 500;
    break;
  case DONE:
    on = t % 3000 < 1500;
    break;
  default:
    on = t % 500 < 250;
  }
  digitalWrite(LED_PIN, on);
}

struct Trial
{
  uint32_t pulses;
  float secs, litres;
};
const int MAX_TRIALS = 12;
Trial trials[MAX_TRIALS];
int nTrials = 0;
bool trialRunning = false;
uint32_t trialStartPulses = 0, trialStartMs = 0;
Trial pending = {};

```

```
void printTrials()
{
  Serial.println("# Trial, Pulses, Time(s), Frequency(Hz), Volume(L), ActualFlow(L/min)");
  for (int i = 0; i < nTrials; i++)
  {
    Trial &t = trials[i];
    Serial.printf("# %d, %lu, %.2f, %.2f, %.3f, %.3f\n", i + 1, (unsigned long)t.pulses,
                  t.secs, t.pulses / t.secs, t.litres, t.litres / t.secs * 60);
  }
}

// least squares Q = a*f + b over the trials
bool fitTrials(float &a, float &b)
{
  if (nTrials < 2)
  {
    Serial.println("# Need at least 2 trials (5 recommended)");
    return false;
  }
  double f[MAX_TRIALS], q[MAX_TRIALS];
  double sf = 0, sq = 0, sff = 0, sfq = 0;
  for (int i = 0; i < nTrials; i++)
  {
    f[i] = trials[i].pulses / trials[i].secs;
    q[i] = trials[i].litres / trials[i].secs * 60;
    sf += f[i];
    sq += q[i];
    sff += f[i] * f[i];
    sfq += f[i] * q[i];
  }
  double n = nTrials, den = n * sff - sf * sf;
  if (fabs(den) < 1e-9)
  {
    Serial.println("# Trials all at the same frequency, cannot fit");
    return false;
  }
  a = (n * sfq - sf * sq) / den;
  b = (sq - a * sf) / n;

  double mean = sq / n, ssRes = 0, ssTot = 0, maxErr = 0, sumErr = 0;
  for (int i = 0; i < nTrials; i++)
  {
    double e = q[i] - (a * f[i] + b);
    ssRes += e * e;
    ssTot += (q[i] - mean) * (q[i] - mean);
    double pct = fabs(e) / q[i] * 100;
    maxErr = max(maxErr, pct);
    sumErr += pct;
  }
  double r2 = ssTot > 0 ? 1 - ssRes / ssTot : 1;
  Serial.printf("# FIT: a = %.6f L/min/Hz, b = %.4f L/min, R^2 = %.5f\n", a, b, r2);
  Serial.printf("# Max calibration error = %.2f %%, average = %.2f %%\n", maxErr, sumErr / n);
  Serial.printf("# (K = %.1f pulses/L, f = %.2f * Q)\n", 60 / a, 1 / a);
  return true;
}

Preferences prefs;
float fitA = 0, fitB = 0;
bool fitValid = false;

void printHelp()
{
  Serial.println(
      "# Commands:\n"
      "#  start | stop | next        run sequence / stop all / skip to next phase\n"
      "#  status                     show state, flow, calibration\n"
      "#  c1time <s> | bvtime <s>    Case 1 duration / BV-02 window\n"
      "#  c2time <s>                 Case 2 duration\n"
      "#  fsraw 1|0                  FT-01 pin level and raw edge count\n"
      "#  hold 0|1                   Case 1: P-02 at 100% (0) or PI to 3 L/min (1)\n"
      "#  log <ms>                   logging period\n"
```

```
      "#  reset                      zero the totaliser\n"
      "#  test                       manual mode (from IDLE):\n"
      "#    p1|p2|p3 <0-100>, sov 0|1, flow <L/min> | flow off\n"
      "#  cal                        show a, b\n"
      "#  seta <a> | setb <b>        set calibration, calsave to keep\n"
      "#  calsave | caldefault       store in flash / back to defaults\n"
      "#  trial start | trial stop | trial vol <L> | trial list | trial clear\n"
      "#  fit | fit save             least-squares Q = a f + b from trials");
}

void handleCommand(String line)
{
  line.trim();
  if (!line.length())
    return;
  int sp = line.indexOf(' ');
  String cmd = sp < 0 ? line : line.substring(0, sp);
  String arg = sp < 0 ? "" : line.substring(sp + 1);
  arg.trim();
  cmd.toLowerCase();

  if (cmd == "help")
  {
    printHelp();
  }
  else if (cmd == "start")
  {
    if (state == IDLE || state == DONE)
      enterState(CASE1);
  }
  else if (cmd == "stop")
  {
    enterState(IDLE);
  }
  else if (cmd == "next")
  {
    if (state == CASE1)
      enterState(FLUSH_PREP);
    else if (state == FLUSH_PREP)
      enterState(FLUSH);
    else if (state == FLUSH)
      enterState(DONE);
  }
  else if (cmd == "status")
  {
    Serial.printf("# state=%s t=%lus flow=%.3f L/min total=%.3f L float=%s sov=%d "
                  "P1=%.0f P2=%.0f P3=%.0f a=%.6f b=%.4f c1time=%lu bvtime=%lu c2time=%lu hold=%d\n",
                  stateName[state], (unsigned long)secondsInState(), flowLpm, totalL,
                  floatHigh ? "HIGH" : "normal", sovOpen, pumps[P1].duty, pumps[P2].duty,
                  pumps[P3].duty, calA, calB, (unsigned long)case1Time,
                  (unsigned long)bv02Window, (unsigned long)case2Time, holdFlow);
  }
  else if (cmd == "c1time")
  {
    case1Time = arg.toInt();
  }
  else if (cmd == "bvtime")
  {
    bv02Window = arg.toInt();
  }
  else if (cmd == "c2time")
  {
    case2Time = arg.toInt();
  }
  else if (cmd == "fsraw")
  {
    fsRaw = arg.toInt() != 0;
    Serial.println(fsRaw ? "# FT-01 raw pin readout ON" : "# FT-01 raw pin readout OFF");
  }
  else if (cmd == "hold")
  {
```

```
    holdFlow = arg.toInt() != 0;
    integral = 0;
    setpoint = TARGET_FLOW;
    if (!holdFlow && state == CASE1)
      setPump(P2, CASE1_DUTY_P2);
  }
  else if (cmd == "log")
  {
    logMs = max(100L, arg.toInt());
  }
  else if (cmd == "reset")
  {
    totalL = 0;
    runPulses = 0;
  }
  else if (cmd == "test")
  {
    if (state == IDLE || state == DONE)
      enterState(TEST);
  }
  else if (state == TEST && (cmd == "p1" || cmd == "p2" || cmd == "p3"))
  {
    int i = cmd[1] - '1';
    if (i == P2)
      piOn = false;
    setPump(i, arg.toFloat());
  }
  else if (state == TEST && cmd == "sov")
  {
    setSolenoid(arg.toInt() != 0);
  }
  else if (state == TEST && cmd == "flow")
  {
    if (arg == "off")
    {
      piOn = false;
      setPump(P2, 0);
    }
    else
    {
      setpoint = arg.toFloat();
      integral = 0;
      piOn = true;
    }
  }
  else if (cmd == "cal")
  {
    Serial.printf("# Q = %.6f * f + %.4f\n", calA, calB);
  }
  else if (cmd == "seta")
  {
    calA = arg.toFloat();
  }
  else if (cmd == "setb")
  {
    calB = arg.toFloat();
  }
  else if (cmd == "calsave")
  {
    prefs.putFloat("a", calA);
    prefs.putFloat("b", calB);
    Serial.println("# calibration saved");
  }
  else if (cmd == "caldefault")
  {
    calA = DEFAULT_A;
    calB = DEFAULT_B;
    prefs.clear();
    Serial.println("# calibration reset to defaults");
  }
  else if (cmd == "trial")
```

```
  {
    if (arg == "start")
    {
      trialStartPulses = readPulses();
      trialStartMs = millis();
      trialRunning = true;
      Serial.println("# trial started: put the return line into the measuring jug");
    }
    else if (arg == "stop" && trialRunning)
    {
      trialRunning = false;
      pending.pulses = readPulses() - trialStartPulses;
      pending.secs = (millis() - trialStartMs) / 1000.0f;
      Serial.printf("# trial stopped: N = %lu pulses, t = %.2f s, f = %.2f Hz. Now enter: trial vol <lit
res collected>\n",
                    (unsigned long)pending.pulses, pending.secs, pending.pulses / pending.secs);
    }
    else if (arg.startsWith("vol") && pending.secs > 0)
    {
      if (nTrials >= MAX_TRIALS)
      {
        Serial.println("# trial table full");
        return;
      }
      pending.litres = arg.substring(3).toFloat();
      trials[nTrials++] = pending;
      pending.secs = 0;
      printTrials();
    }
    else if (arg == "list")
    {
      printTrials();
    }
    else if (arg == "clear")
    {
      nTrials = 0;
      Serial.println("# trials cleared");
    }
  }
  else if (cmd == "fit")
  {
    if (arg == "save" && fitValid)
    {
      calA = fitA;
      calB = fitB;
      prefs.putFloat("a", calA);
      prefs.putFloat("b", calB);
      Serial.println("# fitted calibration applied and saved");
    }
    else if (arg == "save")
    {
      Serial.println("# run 'fit' first");
    }
    else
    {
      fitValid = fitTrials(fitA, fitB);
    }
  }
  else
  {
    Serial.printf("# unknown command '%s' (type help)\n", line.c_str());
  }
}

void pollSerial()
{
  static String buf;
  while (Serial.available())
  {
    char c = Serial.read();
    if (c == '\n' || c == '\r')
```

```
    {
      handleCommand(buf);
      buf = "";
    }
    else if (buf.length() < 80)
    {
      buf += c;
    }
  }
}

void setup()
{
  pinMode(SOLENOID_PIN, OUTPUT);
  setSolenoid(false);
  for (int i = 0; i < 3; i++)
    setupPump(i);
  pinMode(LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT);
  pinMode(FLOAT_PIN, INPUT);
  pinMode(FLOW_PIN, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(FLOW_PIN), onPulse, FALLING);

  Serial.begin(115200);
  delay(200);

  prefs.begin("ft01", false);
  calA = prefs.getFloat("a", DEFAULT_A);
  calB = prefs.getFloat("b", DEFAULT_B);

  Serial.println("# Lam Stage 2 flow skid controller");
  Serial.printf("# FT-01 calibration: Q = %.6f * f + %.4f\n", calA, calB);
  Serial.println("# ON/OFF: start Case 1, press for Case 2, press again to stop. Type 'help' for command
s.");
  Serial.println("H,t_ms,state,pulses,f_Hz,Q_Lmin,Q_mLs,total_L,float_high,p1,p2,p3,sov");

  lastSampleMs = millis();
  lastPulses = readPulses();
  enterState(IDLE);
}

void loop()
{
  pollSerial();

  if (buttonPressed())
  {
    Serial.println("# Button pressed");
    if (state == IDLE || state == DONE)
      enterState(CASE1);
    else if (state == CASE1)
      enterState(FLUSH_PREP);
    else if (state == TEST)
      enterState(IDLE);
    else
      enterState(DONE);
  }

  readFloat();

  if (fsRaw)
  {
    if (digitalRead(FLOW_PIN))
      fsHigh++;
    else
      fsLow++;
  }

  uint32_t now = millis();
  if (now - lastSampleMs >= logMs)
  {
```

```
    float dt = (now - lastSampleMs) / 1000.0f;
    sampleFlow();
    if (state == TEST && piOn)
    {
      runPI(dt);
    }
    else if (state == CASE1 && holdFlow)
    {
      if (p1Held)
        setPump(P2, 100); // sump high: drain as fast as possible
      else
        runPI(dt);
    }
    Serial.printf("D,%lu,%s,%lu,%.2f,%.3f,%.2f,%.4f,%d,%.0f,%.0f,%.0f,%d\n",
                  (unsigned long)now, stateName[state], (unsigned long)runPulses, freqHz,
                  flowLpm, flowLpm * 1000 / 60, totalL, floatHigh, pumps[P1].duty,
                  pumps[P2].duty, pumps[P3].duty, sovOpen);
    if (fsRaw)
    {
      noInterrupts();
      uint32_t raw = rawEdges;
      interrupts();
      uint32_t samples = fsHigh + fsLow;
      Serial.printf("# FS pin=%d high=%lu%% raw_edges=+%lu (total %lu) counted=%lu\n",
                    digitalRead(FLOW_PIN), (unsigned long)(samples ? 100UL * fsHigh / samples : 0),
                    (unsigned long)(raw - fsLastRaw), (unsigned long)raw, (unsigned long)readPulses());
      fsLastRaw = raw;
      fsHigh = fsLow = 0;
    }
  }

  runSequence();
  updatePumps();
  updateLed();
}
```

flow_logger.py

Python · 83 lines

```python
"""Logs the flow skid's serial output to CSV and plots the trends.

Usage:
    pip install pyserial matplotlib
    python flow_logger.py COM5            # Windows (check Device Manager)
    python flow_logger.py /dev/ttyUSB0    # Linux / macOS
Ctrl+C stops logging, saves flow_log_<time>.csv and shows the plots
(flow in L/min and mL/s vs time, totalised litres, pump duties).
Anything typed in the PlatformIO monitor can't be sent while this runs;
close the monitor first, the port can only be opened once.
"""

import csv
import sys
import time

import matplotlib.pyplot as plt
import serial

port = sys.argv[1] if len(sys.argv) > 1 else "/dev/ttyUSB0"
fields = [
    "t_ms",
    "state",
    "pulses",
    "f_Hz",
    "Q_Lmin",
    "Q_mLs",
    "total_L",
    "float_high",
    "p1",
    "p2",
    "p3",
    "sov",
]
rows = []
name = time.strftime("flow_log_%Y%m%d_%H%M%S.csv")

with serial.Serial(port, 115200, timeout=1) as ser, open(name, "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(fields)
    print(
        f"Logging {port} to {name}. Press the ON/OFF button to start; Ctrl+C to finish."
    )
    try:
        while True:
            line = ser.readline().decode(errors="ignore").strip()
            if line.startswith("D,"):
                parts = line[2:].split(",")
                if len(parts) == len(fields):
                    w.writerow(parts)
                    rows.append(parts)
            elif line:
                print(line)
    except KeyboardInterrupt:
        pass

if not rows:
    sys.exit("No data received.")

t0 = float(rows[0][0])
t = [(float(r[0]) - t0) / 1000 for r in rows]
col = lambda k: [float(r[fields.index(k)]) for r in rows]

fig, ax = plt.subplots(4, 1, sharex=True, figsize=(11, 10))
ax[0].plot(t, col("Q_Lmin"))
ax[0].set_ylabel("Flow (L/min)")
ax[0].axhline(3.0, ls="--", c="grey", lw=0.8)
```

```
ax[1].plot(t, col("Q_mLs"), c="tab:orange")
ax[1].set_ylabel("Flow (mL/s)")
ax[2].plot(t, col("total_L"), c="tab:green")
ax[2].set_ylabel("Transferred (L)")
for k in ("p1", "p2", "p3"):
    ax[3].plot(t, col(k), label=k.upper())
ax[3].plot(t, [100 * v for v in col("float_high")], "k:", label="Sump high")
ax[3].set_ylabel("Duty (%)")
ax[3].legend(loc="upper right")
ax[3].set_xlabel("Time (s)")
for a in ax:
    a.grid(alpha=0.3)
fig.suptitle(f"FT-01 flow vs time  (final total {rows[-1][6]} L)")
fig.tight_layout()
fig.savefig(name.replace(".csv", ".png"), dpi=150)
plt.show()
```